

Looks like 'C', but it's not 'C'!

Here is a function for returning the square of a number:

```
func square(f float) float { return f * f }
```

This function definition looks very similar to C, but there are differences. For example, the argument is declared as *f float* and not *float f* as we would write in C. In Go, variable names are used first, followed by the type declaration. Similarly, the function's return type is given after the parameter list; in C, we specify it before the function name. It takes time to get used to these differences, but this approach has resulted in considerably simpler declaration syntax than in C.

In Go, we can return more than one return value. This code segment gets the integer return value from *Atoi* (a string to the integer conversion function in the *strconv* package) function; it also gets the error value, which will be useful to check in case the function fails:

```
str := "10";
i, err := strconv.Atoi(str);
print(i)
```

Multiple return values is a useful feature in Go, particularly for handling error conditions. This code also shows the built-in *print* function to print to the console—Go has a few built-in functions like this.

In Go, many features are similar to each other. We just covered multiple return values—can you now guess (given that *i* and *j* are integers) what this statement does?

```
j, i = i, j;
```

Yes, that's right, it swaps the values in the variables *i* and *j*! Simple, concise and neat, right?

A function example

Here is a more difficult example using functions:

```
func transform(arr []int, foos []func(int) int) {
    for _, foo := range foos {
        for i := 0; i < len(arr); i++ {
            arr[i] = foo(arr[i])
        }
    }
    return
}

func main() {
    arr := []int{1, 2, 3, 4, 5};

    foos := []func(int) int {
        func(arg int) int { return arg * arg },
        func(arg int) int { return -arg }
    };
}
```

```
transform(arr, foos);
```

```
fmt.Println(arr);
}
```

We'll start from the *transform* function. It takes two arguments: an array of integers, and an array of functions (in which each function accepts a single *int* argument, and returns an *int*). In the *for* loop, we use the *range* keyword to get the indexes and values in the array of functions. Since we want only the value here, we ignore the index by assigning it to *_*. Now, each value assigned to the *foo* variable is a function, and we call that function for each element in the integer array, storing the function's return value back in the same element of the integer array.

In the *main* function, we declare a slice (that internally points to an array), and a slice of functions. We create two functions without names ('function literals'), and put them in the slice. We pass these two variables to the *transform* function. The first function literal squares the integer value; the second function literal negates the value. So, this program prints: [-1 -4 -9 -16 -25].

Introducing structs and methods

Here is the declaration of a simple *struct* named 'Point', with two integer members, 'x' and 'y':

```
type Point struct {
    x, y int;
};
```

We can declare methods outside the *struct*. Here is the *Print* method, which prints the values in a *Point* variable:

```
func (p Point) Print() {
    fmt.Printf("(x = %d, y = %d)", p.x, p.y);
}
```

Method definition syntax is different from that of functions—a method definition takes a *struct* type before its name, which is an implicit parameter during the call to that method.

Now, this code in the *main* function creates a *Point* variable, and prints out its value:

```
p := Point{10, 20};
fmt.Printf("%w", p);
p.Print()
```

It prints:

```
{10, 20}
(x = 10, y = 20)
```

Here, *Point{10, 20}*; is a *struct* literal, and looks like a constructor call. We use the '%w' descriptor of the *Printf* family of functions to print the *struct* variable to console, followed by our custom *Print* method's output.